

Artificial Intelligence Agents — Laboratory Report

Constructing a Goal-Based Agent using a Large Language Model

Warehouse Navigation Problem

1. Task 1 — Understanding the Problem

1.1 What is the environment?

The environment is a two-dimensional warehouse grid. Shelving units are represented by # and cannot be crossed. S is the starting position, G is the destination, and . represents free space.

```
#####
#S...#.....G#
#.#.#.....#..#
#...##.....#
#.#.#.#.#.#.#.#
#.....#.....#
#####
```

1.2 What is the goal of the agent?

The goal is to determine a collision-free path from the vehicle's current position S to the destination G.

1.3 What actions are available?

Up, Down, Left, and Right. Each action changes the vehicle's position by one grid square.

1.4 What information must the agent maintain?

The agent needs the warehouse map, current position, goal position, obstacle locations, and search information recording which states have been explored and how each discovered state was reached. The implementation uses a queue, a parent mapping, and an action mapping.

1.5 Why is this goal-based rather than simple reflex?

The agent has an explicit future objective—reaching G—and searches over possible action sequences to achieve that objective. It therefore does more than apply a fixed action directly from the current percept.

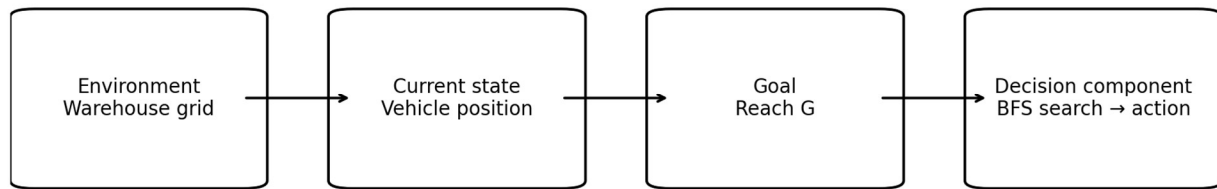
1.6 What changes if the warehouse becomes twice as large?

BFS remains applicable to a finite unweighted grid, but its time and memory requirements grow as more states become reachable. A larger warehouse can require substantially more queue space, explored-state records, and parent information. The appropriate search strategy could depend on the scale and whether shortest-path guarantees remain necessary.

2. Task 2 — Designing the Agent

Component	Design
Environment	Warehouse grid containing free cells and obstacles.
Current state	Vehicle's current row and column.
Goal	Grid position marked G.
Available actions	Up, Down, Left, Right.

Decision-making component	Breadth-First Search over reachable grid states; reconstruct the path from G back to S.
---------------------------	--



Observe state → search toward goal → select action → update state

3. Task 3 — Prompt Engineering

Exact prompt used:

Write a well-documented Python program implementing a goal-based agent for the warehouse navigation problem below.

Warehouse map:

```

#####
#S...#.....G#
#.#...#####.#
#...##.....#
#.#...###.#...#
#.....#.....#
#####
  
```

Requirements:

- 1. Represent the warehouse as a two-dimensional grid.*
- 2. Treat S as the start, G as the goal, # as an obstacle, and . as free space.*
- 3. The vehicle may move only Up, Down, Left, or Right, one grid square at a time.*
- 4. Determine a collision-free path from S to G.*
- 5. Avoid every obstacle and remain inside the grid.*
- 6. Print the path found as positions and/or actions, or print a suitable message if no path exists.*
- 7. Choose and explain an appropriate search algorithm for a goal-based agent.*
- 8. Include a small test that verifies the returned path never enters an obstacle and ends at G.*
- 9. Include a no-path test or otherwise demonstrate the failure case.*
- 10. Keep the implementation self-contained and runnable with standard Python 3, with no external packages.*

3.1 Did the LLM generate a working program on the first attempt?

Yes. The first implementation produced a route for the supplied warehouse. It was then executed and checked using explicit assertions for collision avoidance, legal movement, correct start and goal, and failure handling.

3.2 What search algorithm did it choose?

Breadth-First Search (BFS).

3.3 Why is BFS appropriate here?

Every allowed move has equal cost, so the warehouse can be treated as an unweighted graph. BFS explores states in increasing order of path length and therefore returns a shortest route in number of moves when a route exists.

4. Execution and Validation Results

The supplied map produced a collision-free route containing 20 moves, beginning at (1, 1) and ending at (1, 19).

Test	Result	Evidence
Path exists	PASS	BFS returned a 20-move route.
Correct start	PASS	First state = (1, 1).
Correct goal	PASS	Final state = (1, 19).
Obstacle avoidance	PASS	Every route cell was checked and none was #.
Legal movement	PASS	Every consecutive pair is one allowed grid move.
Shortest path	PASS	BFS route length = 20 moves.
No-path handling	PASS	Blocking both exits from S caused BFS to return no path.

4.1 Found route

Coordinates:

(1, 1) → (1, 2) → (1, 3) → (1, 4) → (2, 4) → (2, 5) → (2, 6) → (2, 7) → (1, 7) → (1, 8) → (1, 9) → (1, 10) → (1, 11) → (1, 12) → (1, 13) → (1, 14) → (1, 15) → (1, 16) → (1, 17) → (1, 18) → (1, 19)

Actions:

Right → Right → Right → Down → Right → Right → Right → Up → Right → Right → Right → Right → Right → Right → Right → Right → Right → Right → Right → Right

5. Final Questions

1. Did the LLM generate a working program on the first attempt?

Yes. The first implementation produced a path and passed the explicit validation tests. Human execution and verification were still necessary.

2. If not, how can you improve your prompt?

The first implementation worked, so no corrective prompt was needed. If correction were necessary, the prompt could be made more explicit about the exact map, action constraints, failure behaviour, and required tests.

3. What search algorithm did the LLM choose?

Breadth-First Search (BFS).

4. Why do you think the LLM selected this algorithm?

The task is an unweighted grid in which every movement costs one step and the goal is to reach a specified destination while avoiding obstacles. BFS systematically explores reachable states and gives a shortest path in number of moves.

7. Conclusion

The warehouse navigation problem is naturally formulated as a goal-based agent: the current position is the state, G is the explicit goal, and movement actions are selected using search. The BFS implementation found a 20-move collision-free shortest route and passed tests for obstacle avoidance, legal movement, start/goal correctness, and the no-path case.